

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum- 590014, Karnataka.



LAB RECORD

on

Big Data Analytics (23CS6PCBDA)

Submitted by

Sushma B T(1WA23CS002)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU - 560019

February 2026 – July 2026

B.M.S. College of Engineering

Bull Temple Road, Bangalore 560019
(Affiliated to Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Big Data Analytics” carried out by **Sushma B T (1WA23CS002)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfilment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025. The Lab report has been approved as it satisfies the academic requirements in respect of a Big Data Analytics (23CS6PCBDA)work prescribed for the said degree.

Anita Harish K
Assistant Professor
Department of CSE, BMSCE

Dr. Kavitha Sooda
Professor & HOD
Department of CSE, BMSCE

INDEX

Sl. No.	Date	Experiment Title	Page No.
1	04.03.26	MongoDB- CRUD Operations Demonstration (Practice and Self Study)	1
2	01.04.26	Perform the following DB operations using Cassandra. • Create a keyspace by name Employee • Create a column family by name ○ Employee-Info with attributes ○ Emp_Id Primary Key, Emp_Name, Designation, ○ Date_of_Joining, Salary, Dept_Name • Insert the values into the table in batch • Update Employee name and Department of EmpId 121 • Sort the details of Employee records based on salary • Alter the schema of the table Employee_Info to add a column Projects which stores a set of Projects done by the corresponding Employee. • Update the altered table to add project names. • Create a TTL of 15 seconds to display the values of Employees.	7
3	08.04.26	Perform the following DB operations using Cassandra. • Create a keyspace by name Library • Create a column family by name Library-Info with attributes ○ Stud_Id Primary Key, ○ Counter_value of type Counter, ○ Stud_Name, Book-Name, Book-Id, ○ Date_of_issue • Insert the values into the table in batch • Display the details of the table created and increase the value of the counter • Write a query to show that a student with id 112 has taken a book “BDA” 2 times. • Export the created column to a csv file • g) Import a given csv dataset from local file system into Cassandra column family	8
4	15.04.26	Execution of HDFS Commands for interaction with Hadoop Environment. (Minimum 10 commands to be executed)	11
5	15.04.26	Implement Wordcount program on Hadoop framework	13
6	06.05.26	From the following link extract the weather data https://github.com/tomwhite/hadoop-book/tree/master/input/ncdc/all • Create a MapReduce program to find average temperature for each year from NCDC data set. • b) find the mean max temperature for every month	16

7	20.05.26	For a given Text file, Create a Map Reduce program to sort the content in an alphabetic order listing only top 10 maximum occurrences of words.	25
8	20.05.26	Write a Scala program to print numbers from 1 to 100 using for loop.	30
9	20.05.26	Using RDD and FlatMap count how many times each word appears in a file and write out a list of words whose count is strictly greater than 4 using Spark.	31
10	20.05.26	Write a simple streaming program in Spark to receive text data streams on a particular port, perform basic text cleaning (like white space removal, stop words removal, lemmatization, etc.), and print the cleaned text on the screen. (Open Ended Question).	32

Github Link: <https://github.com/Yukta-bmsce/BDA>

Course Outcomes (COs):

CO1	Apply the concept of NoSQL, Hadoop or Spark for a given task
CO2	Analyse big data analytics mechanisms that can be applied to obtain solution for a given problem.
CO3	Design and implement solutions using data analytics mechanisms for a given problem.

LABORATORY PROGRAM – 1

MongoDB- CRUD Operations Demonstration (Practice and Self Study)

COMMAND WITH OUTPUT - USING ATLAS

```
C:\Users\student>mongoimport
2025-03-04T15:04:25.938+0530 no collection specified
2025-03-04T15:04:25.938+0530 using filename '' as collection
2025-03-04T15:04:25.938+0530 error validating settings: invalid collection name: collection name cannot be an empty string

C:\Users\student>mongoexport
2025-03-04T15:04:49.930+0530 must specify a collection
2025-03-04T15:04:49.931+0530 try 'mongoexport --help' for more information

C:\Users\student>mongoexport mongodb+srv://uzairobaid:uzairobaid123@cluster0.pdibg.mongodb.net/DBMS_DEMO --collection=Student --out C:\Users\student\Downloads\output.json
2025-03-04T15:11:46.757+0530 connected to: mongodb+srv://[**REDACTED**]@cluster0.pdibg.mongodb.net/DBMS_DEMO
2025-03-04T15:11:46.979+0530 exported 6 records

Atlas atlas-herlbh-shard-0 [primary] DBMS_DEMO> db.New_Student.find()
[
  {
    _id: ObjectId('67c6c2b14b7583e62cfa4215'),
    RollNo: 2,
    Age: 22,
    Cont: 9976,
    email: 'anushka.de9@gmail.com'
  },
  {
    _id: ObjectId('67c6c2c84b7583e62cfa4216'),
    RollNo: 3,
    Age: 21,
    Cont: 5576,
    email: 'anubhav.de9@gmail.com'
  },
  {
    _id: ObjectId('67c6c2c74b7583e62cfa4217'),
    RollNo: 4,
    Age: 20,
    Cont: 4476,
    email: 'pani.de9@gmail.com'
  },
  {
    _id: ObjectId('67c6c2cd4b7583e62cfa4218'),
    RollNo: 10,
    Age: 23,
    Cont: 2276,
    email: 'Abhinav@gmail.com'
  },
  {
    _id: ObjectId('67c6c3ac4b7583e62cfa4219'),
    RollNo: 11,
    Age: 22,
    Name: 'FEH',
    Cont: 2276,
    email: 'rea.de9@gmail.com'
  },
  {
    _id: ObjectId('67c6c27b4b7583e62cfa4214'),
    RollNo: 1,
    Age: 21,
    Cont: 9876,
    email: 'antara.de9@gmail.com'
  }
]
```

a) DATABASE IN MONGODB.

CREATEuse myDB;

Confirm the existence of your database

db;

To list all databases **show
dbs;**

b) **CRUD (CREATE, READ, UPDATE, DELETE) OPERATIONS**

- To create a collection by the name “Student”. Let us take a look at the collection list prior to the creation of the new collection “Student”. **db.createCollection(“Student”);**

- To drop a collection by the name “Student”.

db.Student.drop();

- Create a collection by the name “Students” and store the following data in it.

db.Student.insert({_id:1,StudName:"MichelleJacintha",Grade:"VII",Hobbies:"InternetSurfing"});

- Insert the document for “AryanDavid” in to the Students collection only if it does not already exist in the collection.

db.Student.update({_id:3,StudName:"AryanDavid",Grade:"VII"},{\$set:{Hobbies:"Skating"}},{upsert:true});

- **FIND METHOD**

- To search for documents from the “Students” collection based on certain search criteria.

db.Student.find({StudName:"Aryan David"});

- To display only the StudName and Grade from all the documents of the Students collection. The identifier _id should be suppressed and NOT displayed.

db.Student.find({}, {StudName:1,Grade:1, _id:0});

- To find those documents where the Grade is set to ‘VII’
db.Student.find({Grade:{Seq:'VII'}}).pretty();

- To find those documents from the Students collection where the Hobbies is set to either ‘Chess’ or is set to ‘Skating’.

db.Student.find({Hobbies :{ \$in: ['Chess','Skating']}}).pretty ();

- To find documents from the Students collection where the StudName begins with “M”.

db.Student.find({StudName:/^M/}).pretty();

- To find documents from the Students collection where the StudName has an “e” in any position.

db.Student.find({StudName:/e/}).pretty();

- To find the number of documents in the Students collection. **db.Student.count();**

- To sort the documents from the Students collection in the descending order of StudName.

db.Student.find().sort({StudName:-1}).pretty();

c) **Import data from a CSV file**

Given a CSV file “sample.txt” in the D:drive, import the file into the MongoDB collection, “SampleJSON”. The collection is in the database “test”.

mongoimport --db Student --collection airlines --type csv --headerline --file /home/hduser/Desktop/airline.csv

d) **Export data to a CSV file**

This command used at the command prompt exports MongoDB JSON documents from “Customers” collection in the “test” database into a CSV file “Output.txt” in the D:drive. **mongoexport --host localhost --db Student --collection airlines --csv --out /home/hduser/Desktop/output.txt --fields “Year”, “Quarter”**

e) **Save Method :**

Save() method will insert a new document, if the document with the _id does not exist. If it exists it will replace the existing document:

```
db.Students.save({StudName:"Vamsi", Grade:"VI"})
```

f) **Add a new field to existing Document:**

```
db.Students.update({_id:4},{ $set: {Location:"Network"}})
```

g) **Remove the field in an existing Document**

```
db.Students.update({_id:4},{ $unset: {Location:"Network"}})
```

h) **Finding Document based on search criteria suppressing few fields**

```
db.Student.find({_id:1},{StudName:1,Grade:1,_id:0});
```

To find those documents where the Grade is not set to 'VII'

```
db.Student.find({Grade: {$ne:'VII'}}).pretty();
```

To find documents from the Students collection where the StudName ends with s.

```
db.Student.find({StudName:/s$/}).pretty();
```

i) **to set a particular field value to NULL**

```
db.Students.update({_id:3},{ $set: {Location:null}})
```

j) **Count the number of documents in Student Collections**

```
db.Students.count()
```

k) **Count the number of documents in Student Collections with grade :VII**

```
db.Students.count({Grade:"VII"})
```

retrieve first 3 documents

```
db.Students.find({Grade:"VII"}).limit(3).pretty();
```

Sort the document in Ascending order

```
db.Students.find().sort({StudName:1}).pretty();
```

to Skip the 1st two documents from the Students Collections

```
db.Students.find().skip(2).pretty()
```

l) **Create a collection by name "food" and add to each document add a "fruits" array**

```
db.food.insert( { _id:1,
```

```
fruits:['grapes','mango','apple']
```

```
 } ) db.food.insert( { _id:2,
```

```
fruits:['grapes','mango','cherry'] } ) db.food.insert( {
```

```
_id:3, fruits:['banana','mango'] } )
```

To find those documents from the "food" collection which has the "fruits array" constitute of "grapes", "mango" and "apple".

```
db.food.find ( {fruits: ['grapes','mango','apple'] } ). pretty().
```

To find in "fruits" array having "mango" in the first index position.

```
db.food.find ( {'fruits.1':'grapes'} )
```

To find those documents from the "food" collection where the size of the array is two.

```
db.food.find ( {"fruits": {$size:2}} )
```

To find the document with a particular id and display the first two elements from the array “fruits”
`db.food.find({_id:1},{“fruits”:$slice:2})`

To find all the documents from the food collection which have elements mango and grapes in the array “fruits”

`db.food.find({fruits:{$all:[“mango”,“grapes”]}})`

update on Array:

using particular id replace the element present in the 1st index position of the fruits array with apple

`db.food.update({_id:3},{set:{fruits.1:'apple'}})`

insert new key value pairs in the fruits array

`db.food.update({_id:2},{push:{price:{grapes:80,mango:200,cherry:100}}})`

XII. Aggregate Function :

Create a collection Customers with fields custID, AcctBal, AcctType. Now group on

“custID” and compute the sum of “AccBal”. `db.Customers.aggregate ( {$group : { _id : “$custID”,TotAccBal : {$sum:”$AccBal”} } } );`

match on AcctType:”S” then group on “CustID” and compute the sum of “AccBal”.

`db.Customers.aggregate ( {$match:{AcctType:”S”}},{$group : { _id : “$custID”,TotAccBal : {$sum:”$AccBal”} } } );`

match on AcctType:”S” then group on “CustID” and compute the sum of “AccBal” and total balance greater than 1200. `db.Customers.aggregate ( {$match:{AcctType:”S”}},{$group : { _id : “$custID”,TotAccBal :`

`{$sum:”$AccBal”} } },`

`{$match:{TotAccBal:{$gt:1200}}});`

```
C:\Users\student>mongoimport mongodb+srv://uzairobaid:uzairobaid123@cluster0.pdibg.mongodb.net/DBMS_DEMO --collection=New_Student --type json --file C:\Users\student\Downloads\output.json
2025-03-04T15:19:31.071+0530    connected to: mongodb+srv://[**REDACTED**]@cluster0.pdibg.mongodb.net/DBMS_DEMO
2025-03-04T15:19:31.168+0530    6 document(s) imported successfully, 0 document(s) failed to import.
```

```
C:\Users\student>mongoimport mongodb+srv://uzairobaid:uzairobaid123@cluster0.pdibg.mongodb.net/DBMS_DEMO --collection=New_Student --type json --file C:\Users\student\Downloads\output.json
2025-03-04T15:19:31.071+0530    connected to: mongodb+srv://[**REDACTED**]@cluster0.pdibg.mongodb.net/DBMS_DEMO
2025-03-04T15:19:31.168+0530    6 document(s) imported successfully, 0 document(s) failed to import.
```

```
bmscscse@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC: $ mongosh
Current Mongosh Log ID: 67c7f7c16bfaaa2811db83af
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.2.0
Using MongoDB:      7.0.6
Using Mongosh:      2.2.0

For mongosh info see: https://docs.mongodb.com/mongodb-shell/

-----
The server generated these startup warnings when booting
2025-03-11T14:00:00.317+05:30: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2025-03-11T14:00:08.134+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

Enterprise test> use uzairDB
switched to db uzairDB
Enterprise uzairDB> db.createCollection('Student')
{ ok: 1 }
```

```

Enterprise uzairDB> db.Student.insert({_id: 1, StudName: "Michelle Jacintha", Grade: "VII", Hobbies: "Internet Surfing"});
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{ acknowledged: true, insertedIds: { '0': 1 } }
Enterprise uzairDB> db.Student.update(
...   {_id: 3, StudName: "Aryan David", Grade: "VII"},
...   {$set: {Hobbies: "Skating"}},
...   {upsert: true}
... );
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: 3,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
Enterprise uzairDB> db.Student.find({StudName: "Aryan David"});
[
  { _id: 3, StudName: 'Aryan David', Grade: 'VII', Hobbies: 'Skating' }
]
Enterprise uzairDB> db.Student.find({}, {StudName: 1, Grade: 1, _id: 0});
[
  { StudName: 'Michelle Jacintha', Grade: 'VII' },
  { StudName: 'Aryan David', Grade: 'VII' }
]
Enterprise uzairDB> db.Student.find({Grade: {$eq: 'VII'}}).pretty();
[
  {
    _id: 1,
    StudName: 'Michelle Jacintha',
    Grade: 'VII',
    Hobbies: 'Internet Surfing'
  },
  { _id: 3, StudName: 'Aryan David', Grade: 'VII', Hobbies: 'Skating' }
]
Enterprise uzairDB> db.Student.find({Hobbies: { $in: ['Chess', 'Skating']}}).pretty();
[
  { _id: 3, StudName: 'Aryan David', Grade: 'VII', Hobbies: 'Skating' }
]

```

```

Enterprise uzairDB> db.Student.find({StudName: /^M/}).pretty();
[
  {
    _id: 1,
    StudName: 'Michelle Jacintha',
    Grade: 'VII',
    Hobbies: 'Internet Surfing'
  }
]
Enterprise uzairDB> db.Student.find({StudName: /e/}).pretty();
[
  {
    _id: 1,
    StudName: 'Michelle Jacintha',
    Grade: 'VII',
    Hobbies: 'Internet Surfing'
  }
]
Enterprise uzairDB> db.Student.count();
DeprecationWarning: Collection.count() is deprecated. Use countDocuments or estimatedDocumentCount.
2
Enterprise uzairDB> db.Student.find().sort({StudName:-1}).pretty();
[
  {
    _id: 1,
    StudName: 'Michelle Jacintha',
    Grade: 'VII',
    Hobbies: 'Internet Surfing'
  },
  { _id: 3, StudName: 'Aryan David', Grade: 'VII', Hobbies: 'Skating' }
]

```

```
bmscecse@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ cd home
bash: cd: home: No such file or directory
bmscecse@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ cd Desktop
bmscecse@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ touch out.csv
```

```
bmscecse@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ mongoexport --host localhost --db uzairDB --collection Student --type csv --out /home/bmscecse/Desktop/out.csv --fields _id,StudName
2025-03-11T15:01:48.547+0530    connected to: mongodb://localhost/
2025-03-11T15:01:48.549+0530    exported 2 records
```

```
bmscecse@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ mongoimport --db uzairDB --collection Student --type csv --headerline --file /home/bmscecse/Desktop/out.csv --upsert
2025-03-11T15:08:12.504+0530    connected to: mongodb://localhost/
2025-03-11T15:08:12.514+0530    2 document(s) imported successfully. 0 document(s) failed to import.
```

```
Enterprise uzairDB> db.Student.find()
[
  { _id: 1, StudName: 'Michelle Jacintha' },
  { _id: 3, StudName: 'Aryan David' }
]
```

LABORATORY PROGRAM – 2

Perform the following DB operations using Cassandra

Questions:

- m) Create a keyspace by name Employee
- n) Create a column family by name
 - Employee-Info with attributes
 - Emp_Id Primary Key, Emp_Name,
 - Designation, Date_of_Joining,
 - Salary, Dept_Name
- o) Insert the values into the table in batch
- p) Update Employee name and Department of Emp-Id 121
- q) Sort the details of Employee records based on salary
- r) Alter the schema of the table Employee_Info to add a column Projects which stores a set of Projects done by the corresponding Employee.
- s) Update the altered table to add project names.
- t) Create a TTL of 15 seconds to display the values of Employees.

COMMAND WITH OUTPUT

```
cqlsh> CREATE KEYSPACE Employee
... WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
cqlsh> USE Employee;
cqlsh:employee> CREATE TABLE Employee_Info (
...     Emp_Id int PRIMARY KEY,
...     Emp_Name text,
...     Designation text,
...     Date_of_Joining date,
...     Salary float,
...     Dept_Name text
... );
cqlsh:employee> BEGIN BATCH
... INSERT INTO Employee_Info (Emp_Id, Emp_Name, Designation, Date_of_Joining, Salary, Dept_Name)
... VALUES (121, 'Amit', 'Manager', '2018-02-01', 70000.0, 'Sales');
... INSERT INTO Employee_Info (Emp_Id, Emp_Name, Designation, Date_of_Joining, Salary, Dept_Name)
... VALUES (122, 'Priya', 'Developer', '2020-06-15', 50000.0, 'IT');
... INSERT INTO Employee_Info (Emp_Id, Emp_Name, Designation, Date_of_Joining, Salary, Dept_Name)
... VALUES (123, 'Rahul', 'Analyst', '2019-11-20', 60000.0, 'Finance');
... APPLY BATCH;
cqlsh:employee> UPDATE Employee_Info
... SET Emp_Name = 'Amit Kumar', Dept_Name = 'Marketing'
... WHERE Emp_Id = 121;
cqlsh:employee> SELECT * FROM Employee_Info
... WHERE Salary IS NOT NULL
... ALLOW FILTERING;
InvalidRequest: Error from server: code=2200 [Invalid query] message="Unsupported restriction: salary IS NOT NULL"
cqlsh:employee> SELECT * FROM Employee_Info
... WHERE Salary > 0
... ALLOW FILTERING;

emp_id | date_of_joining | dept_name | designation | emp_name | salary
-----|-----|-----|-----|-----|-----
123 | 2019-11-20 | Finance | Analyst | Rahul | 60000
122 | 2020-06-15 | IT | Developer | Priya | 50000
121 | 2018-02-01 | Marketing | Manager | Amit Kumar | 70000
(3 rows)
cqlsh:employee> ALTER TABLE Employee_Info ADD Projects set<text>;
cqlsh:employee> UPDATE Employee_Info
... SET Projects = {'ProjectA', 'ProjectB'}
... WHERE Emp_Id = 121;
cqlsh:employee> INSERT INTO Employee_Info (Emp_Id, Emp_Name, Designation, Date_of_Joining, Salary, Dept_Name)
... VALUES (124, 'Neha', 'HR', '2022-03-01', 45000.0, 'HR')
... USING TTL 15;
cqlsh:employee> SELECT * FROM Employee_Info;

emp_id | date_of_joining | dept_name | designation | emp_name | projects | salary
-----|-----|-----|-----|-----|-----|-----
123 | 2019-11-20 | Finance | Analyst | Rahul | null | 60000
122 | 2020-06-15 | IT | Developer | Priya | null | 50000
121 | 2018-02-01 | Marketing | Manager | Amit Kumar | {'ProjectA', 'ProjectB'} | 70000
(3 rows)
```

LABORATORY PROGRAM – 3

Perform the following DB operations using Cassandra

Questions:

- Create a keyspace by name Library
- Create a column family by name Library-Info with attributes
 - Stud_Id Primary Key,
 - Counter_value of type Counter,
 - Stud_Name, Book-Name, Book-Id,
 - Date_of_issue
- Insert the values into the table in batch
- Display the details of the table created and increase the value of the counter
- Write a query to show that a student with id 112 has taken a book “BDA” 2 times.
- Export the created column to a csv file
- Import a given csv dataset from local file system into Cassandra column family

COMMAND WITH OUTPUT

```
cqlsh:employee> CREATE KEYSPACE IF NOT EXISTS Library
... WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
cqlsh:employee> USE Library;
cqlsh:library> CREATE TABLE IF NOT EXISTS Library_Info (
...     Stud_Id INT PRIMARY KEY,
...     Stud_Name TEXT,
...     Book_Name TEXT,
...     Book_Id TEXT,
...     Date_of_issue DATE
... );
cqlsh:library> CREATE TABLE IF NOT EXISTS Book_Counter (
...     Stud_Id INT,
...     Book_Name TEXT,
...     Counter_value COUNTER,
...     PRIMARY KEY ((Stud_Id), Book_Name)
... );
cqlsh:library> BEGIN BATCH
... INSERT INTO Library_Info (Stud_Id, Stud_Name, Book_Name, Book_Id, Date_of_issue)
... VALUES (112, 'Anjali Rao', 'BDA', 'B101', '2024-10-01');
...
... INSERT INTO Library_Info (Stud_Id, Stud_Name, Book_Name, Book_Id, Date_of_issue)
... VALUES (113, 'Karthik N', 'AI', 'B102', '2024-11-11');
... APPLY BATCH;
cqlsh:library> UPDATE Book_Counter SET Counter_value = Counter_value + 1 WHERE Stud_Id = 112 AND Book_Name = 'BDA';
cqlsh:library> UPDATE Book_Counter SET Counter_value = Counter_value + 1 WHERE Stud_Id = 112 AND Book_Name = 'BDA';
cqlsh:library> SELECT * FROM Book_Counter WHERE Stud_Id = 112 AND Book_Name = 'BDA';

stud_id | book_name | counter_value
-----+-----+-----
112    | BDA      | 4
(1 rows)
```

```

cqlsh> CREATE KEYSPACE Students WITH REPLICATION =
... ('class': 'SimpleStrategy', 'replication_factor': '1');
cqlsh>
cqlsh> USE Students;
cqlsh:students> DESCRIBE KEYSPACES;

companies | library | products | system | system_traces
company | pro | productss | system_auth | system_views
employe | prod | productsss | system_distributed | system_virtual_schema
employee | productname | students | system_schena

cqlsh:students> CREATE TABLE Students_Info (
... Roll_No int PRIMARY KEY,
... StudName text,
... DateOfJoining timestamp,
... last_exam_Percent double
... );
cqlsh:students> SELECT * FROM system.schena_keyspaces;
InvalidRequest: Error from server: code=2200 [Invalid query] message="table schena_keyspaces does not exist"
cqlsh:students>
cqlsh:students> SELECT * FROM system.schena_keyspaces;

keyspace_name | durable_writes | replication
-----
companies | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')
system_auth | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')
system_schena | True | ('class': 'org.apache.cassandra.locator.LocalStrategy')
library | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')
products | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')
system_distributed | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '3')
system | True | ('class': 'org.apache.cassandra.locator.LocalStrategy')
productsss | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')
prod | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')
pro | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')
system_traces | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '2')
students | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')
company | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')
employe | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')
productname | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')
employe | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')
productss | True | ('class': 'org.apache.cassandra.locator.SimpleStrategy', 'replication_factor': '1')

(17 rows)
cqlsh:students> DESCRIBE TABLES;

students_info

```

```

cqlsh:students> SELECT * FROM Students_Info WHERE Roll_No IN (1,2,3);

roll_no | dateofjoining | last_exam_percent | studname
-----+-----+-----+-----
1 | 2012-03-11 18:30:00.000000+0000 | 79.9 | Asha
2 | 2012-03-11 18:30:00.000000+0000 | 89.9 | Kiron
3 | 2012-03-11 18:30:00.000000+0000 | 90.9 | Shanthi

(3 rows)
cqlsh:students> CREATE INDEX ON Students_Info (StudName);
cqlsh:students> SELECT * FROM Students_Info WHERE StudName = 'Asha';

roll_no | dateofjoining | last_exam_percent | studname
-----+-----+-----+-----
1 | 2012-03-11 18:30:00.000000+0000 | 79.9 | Asha

(1 rows)
cqlsh:students> SELECT Roll_No, StudName FROM Students_Info LIMIT 2;

roll_no | studname
-----+-----
5 | Rohan
1 | Asha

(2 rows)
cqlsh:students> SELECT Roll_No AS USN FROM Students_Info;

USN
---
5
1
2
4
3

(5 rows)
cqlsh:students> UPDATE Students_Info
... SET StudName = 'David Sheen'
... WHERE Roll_No = 2;
cqlsh:students> UPDATE Students_Info SET Roll_No = 6 WHERE Roll_No = 3; -- ✖ ERROR!
InvalidRequest: Error from server: code=2200 (invalid query) message="PRIMARY KEY part roll_no found in SET part"

```

```

cqlsh:students> DELETE Last_Exam_Percent FROM Students_Info WHERE Roll_No = 2;
cqlsh:students> DELETE FROM Students_Info WHERE Roll_No = 2;
cqlsh:students> ALTER TABLE Students_Info ADD hobbies SET<text>;
cqlsh:students> ALTER TABLE Students_Info ADD languages LIST<text>;
cqlsh:students> UPDATE Students_Info
... SET hobbies = hobbies + ['Chess', 'Table Tennis']
... WHERE Roll_No = 1;
cqlsh:students> CREATE TABLE library_book (
... counter_value counter,
... book_name text,
... stud_name text,
... PRIMARY KEY(book_name, stud_name)
... );
cqlsh:students> UPDATE library_book
... SET counter_value = counter_value + 1
... WHERE book_name = 'Big Data Analytics' AND stud_name = 'Jeet';
cqlsh:students> CREATE TABLE userlogin (
... userid int PRIMARY KEY,
... password text
... );
cqlsh:students> INSERT INTO userlogin (userid, password)
... VALUES (1, 'lnfy') USING TTL 30;
cqlsh:students> SELECT TTL(password) FROM userlogin WHERE userid = 1;

ttl(password)
-----
20

(1 rows)
cqlsh:students> COPY Students_Info TO '/home/bmsccse/Desktop/Student_Info.csv';
Using 16 child processes

Starting copy of students.students.info with columns [roll_no, dateofjoining, hobbies, languages, last_exam_percent, studname].
Processed: 4 rows; Rate: 38 rows/s; Avg. rate: 38 rows/s
4 rows exported to 1 files in 0.124 seconds.
cqlsh:students> COPY Students_Info FROM '/home/bmsccse/Desktop/Student_Info.csv';
Using 16 child processes

Starting copy of students.students.info with columns [roll_no, dateofjoining, hobbies, languages, last_exam_percent, studname].
Processed: 4 rows; Rate: 7 rows/s; Avg. rate: 11 rows/s
4 rows imported from 1 files in 0.377 seconds (0 skipped).
cqlsh:students> COPY person (id, fname, lname) FROM STDIN;
id,fname,lname not found
cqlsh:students> COPY Students_Info TO STDOUT;
5,2012-03-11 18:30:00.000+0000,,56.9,Rohan
1,2012-03-11 18:30:00.000+0000,['Chess', 'Table Tennis'],79.9,Asha
4,2012-03-11 18:30:00.000+0000,,67.9,Smith
3,2012-03-11 18:30:00.000+0000,,90.9,Shanthi
cqlsh:students>

```


LABORATORY PROGRAM – 4

Execution of HDFS Commands for interaction with Hadoop Environment. (Minimum 10 commands to be executed)

COMMAND WITH OUTPUT

```
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC: $ cd ./Desktop/
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ start-all.sh
WARNING: Attempting to start all Apache Hadoop daemons as hadoop in 10 seconds.
WARNING: This is not a recommended production deployment configuration.
WARNING: Use CTRL-C to abort.
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC]
Starting resourcemanager
Starting nodemanagers
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -mkdir /Lab05
```

```
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Hadoop
ls: '/Hadoop': No such file or directory
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Lab05
```

```
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ touch test.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ nano test.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -put ./text.txt /Lab05/text.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Lab05
Found 1 items
-rw-r--r-- 1 hadoop supergroup          19 2024-05-13 14:33 /Lab05/text.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -cat /Lab05/text.txt
Hello
How are you?
```

```
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Lab05
Found 2 items
-rw-r--r-- 1 hadoop supergroup          15 2024-05-13 14:40 /Lab05/test.txt
-rw-r--r-- 1 hadoop supergroup          19 2024-05-13 14:33 /Lab05/text.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -getmerge /Lab05 /text.txt /Lab05 /test.txt ../Downloads/Merged.txt
getmerge: '/text.txt': No such file or directory
getmerge: '/test.txt': No such file or directory
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -getmerge /Lab05/text.txt /Lab05/test.txt ../Downloads/Merged.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -getfacl /Lab05
# file: /Lab05
# owner: hadoop
# group: supergroup
user::rwx
group::r-x
other::r-x
```

```
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -put /home/hadoop/Desktop/Welcome.txt /abc/WC.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -copyFromLocal /home/hadoop/Desktop/Welcome.txt /abc/WC.txt
copyFromLocal: '/abc/WC.txt': File exists
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -get /abc/WC.txt /home/hadoop/Downloads/WC.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -getmerge /abc/ /home/hadoop/Desktop/Merge.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -getfacl /abc/
# file: /abc
# owner: hadoop
# group: supergroup
user::rwx
group::r-x
other::r-x
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -cat /abc/WC.txt
hello world
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -mv /abc /FFF
hadoop fs -ls /FFF
Found 3 items
-rw-r--r-- 1 hadoop supergroup          12 2025-04-15 14:53 /FFF/WC.txt
-rw-r--r-- 1 hadoop supergroup          12 2024-05-14 14:35 /FFF/file.txt
-rw-r--r-- 1 hadoop supergroup          12 2024-05-14 14:38 /FFF/file_cp_local.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -cp /CSE/ /LLL
```

```
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -copyToLocal /Lab05/text.txt ../Documents
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -copyToLocal /Lab05/text.txt ../Documents
```

```
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -cat /Lab05/text.txt
Hello
How are you?
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -mv /Lab05 /test_Lab05
```

```
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -ls /test_Lab05
Found 2 items
-rw-r--r-- 1 hadoop supergroup 15 2024-05-13 14:40 /test_Lab05/test.txt
-rw-r--r-- 1 hadoop supergroup 19 2024-05-13 14:33 /test_Lab05/text.txt
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -cp /test_Lab05/ /Lab05
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -ls /Lab05
Found 2 items
-rw-r--r-- 1 hadoop supergroup 15 2024-05-13 14:51 /Lab05/test.txt
-rw-r--r-- 1 hadoop supergroup 19 2024-05-13 14:51 /Lab05/text.txt
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -ls /test_Lab05
Found 2 items
-rw-r--r-- 1 hadoop supergroup 15 2024-05-13 14:40 /test_Lab05/test.txt
-rw-r--r-- 1 hadoop supergroup 19 2024-05-13 14:33 /test_Lab05/text.txt
```

LABORATORY PROGRAM – 5

Implement Wordcount program on Hadoop framework

CODE, COMMAND WITH OUTPUT

Driver Code

```
// Importing libraries import java.io.IOException;
import org.apache.hadoop.conf.Configured; import
org.apache.hadoop.fs.Path; import
org.apache.hadoop.io.IntWritable; import
org.apache.hadoop.io.Text; import
org.apache.hadoop.mapred.FileInputFormat; import
org.apache.hadoop.mapred.FileOutputFormat; import
org.apache.hadoop.mapred.JobClient; import
org.apache.hadoop.mapred.JobConf; import
org.apache.hadoop.util.Tool; import
org.apache.hadoop.util.ToolRunner; public class WCDriver
extends Configured implements Tool {

    public int run(String[] args) throws IOException { if
        (args.length < 2) {
            System.out.println("Please give valid inputs"); return
            -1;
        }

        JobConf conf = new JobConf(WCDriver.class);
        conf.setJobName("WordCount");

        FileInputFormat.setInputPaths(conf, new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new Path(args[1]));

        conf.setMapperClass(WCMapper.class);
        conf.setReducerClass(WCReducer.class);

        conf.setMapOutputKeyClass(Text.class);
        conf.setMapOutputValueClass(IntWritable.class);

        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);

        JobClient.runJob(conf);
        return 0;
    }

    // Main Method
    public static void main(String[] args) throws Exception {
        int exitCode = ToolRunner.run(new WCDriver(), args);
        System.out.println("Job Exit Code: " + exitCode); }
}
```

Mapper Code

```
// Importing libraries import
java.io.IOException; import
org.apache.hadoop.io.IntWritable; import
org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import
org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper; import
org.apache.hadoop.mapred.OutputCollector; import
org.apache.hadoop.mapred.Reporter;

public class WCMapper extends MapReduceBase implements Mapper<LongWritable, Text, Text, IntWritable> {
```

```

// Map function
public void map(LongWritable key, Text value, OutputCollector<Text, IntWritable> output, Reporter reporter) throws
    IOException {
    String line = value.toString();

    // Splitting the line on whitespace for (String word :
    line.split("\\s+")) { if (word.length() > 0) {
    output.collect(new Text(word), new IntWritable(1)); }
    }
}
}

```

Reducer Code

```

// Importing libraries import
java.io.IOException;
import java.util.Iterator;

import org.apache.hadoop.io.IntWritable; import
org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer; import
org.apache.hadoop.mapred.Reporter;

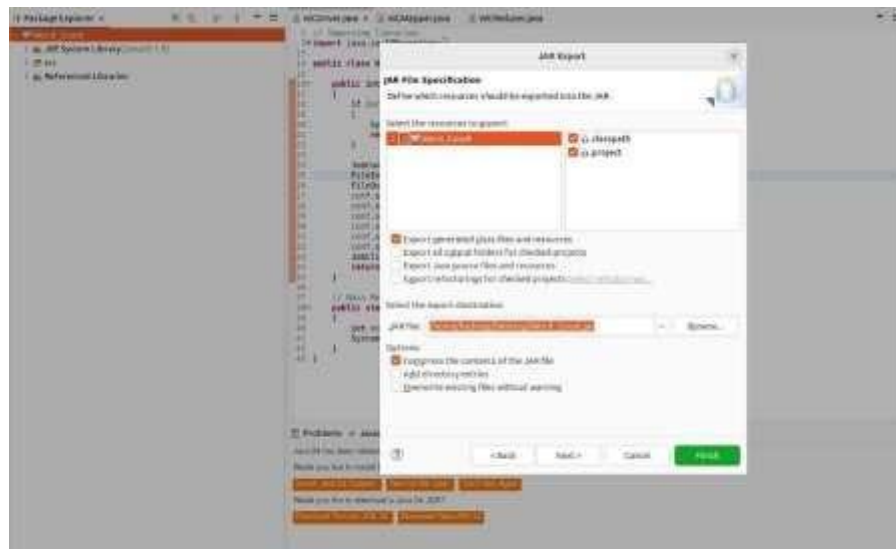
public class WCReducer extends MapReduceBase implements Reducer<Text, IntWritable, Text, IntWritable> {

    // Reduce function
    public void reduce(Text key, Iterator<IntWritable> values,
        OutputCollector<Text, IntWritable> output,
        Reporter reporter) throws IOException { int
        count = 0;

        // Counting the frequency of each word
        while (values.hasNext()) { count +=
        values.next().get();
        }

        output.collect(key, new IntWritable(count));
    }
}

```



```

hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ cd ./Desktop/
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ start-all.sh
WARNING: Attempting to start all Apache Hadoop daemons as hadoop in 10 seconds.
WARNING: This is not a recommended production deployment configuration.
WARNING: Use CTRL-C to abort.
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC]
Starting resourcemanager
Starting nodemanagers
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hdfs dfs -mkdir /Lab06
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /Lab06

```

```

hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ jps
7360 DataNode
7928 ResourceManager
8681 Jps
7178 NameNode
8091 NodeManager
7644 SecondaryNameNode
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ cd ..
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ cd ./Desktop/
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ nano file1.txt

```

```

hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -copyFromLocal -f /home/hadoop/Desktop/file1.txt /rgs/test.txt
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop jar /home/hadoop/Desktop/WordCount.jar wordcount.WordCount /rgs/test.txt /output
JAR does not exist or is not a normal file: /home/hadoop/Desktop/WordCount.jar
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop jar /home/hadoop/Desktop/Word_Count.jar wordcount.WordCount /rgs/test.txt /output
Exception in thread "main" java.lang.ClassNotFoundException: wordcount.WordCount
    at java.base/java.net.URLClassLoader.findClass(URLClassLoader.java:476)
    at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:594)
    at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:527)
    at java.base/java.lang.Class.forName0(Native Method)
    at java.base/java.lang.Class.forName(Class.java:396)
    at org.apache.hadoop.util.RunJar.run(RunJar.java:321)
    at org.apache.hadoop.util.RunJar.main(RunJar.java:241)
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -cat /output/part-00000
are 1
brother 1
family 1
hi 1
how 5
is 4
job 1
sister 1
you 1
your 4
hadoop@bmscsecse-HP-Elite-Tower-800-G9-Desktop-PC:~/Desktop$ hadoop fs -ls /output
Found 2 items
-rw-r--r-- 1 hadoop supergroup 0 2024-05-21 15:21 /output/SUCCESS
-rw-r--r-- 1 hadoop supergroup 69 2024-05-21 15:21 /output/part-00000

```

LABORATORY PROGRAM – 6

Implement Weather program on Hadoop framework

Questions:

From the following link extract the weather data

<https://github.com/tomwhite/hadoopbook/tree/master/input/ncdc/all>

- Create a MapReduce program to find average temperature for each year from NCDC data set.
- find the mean max temperature for every month.

CODE, COMMAND WITH OUTPUT – A

Driver Code

```
package temp;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AverageDriver { public static void main(String[] args)
throws Exception {

    if (args.length != 2) {
        System.err.println("Please enter both input and output parameters.");
        System.exit(-1);
    }

    // Creating a configuration and job instance
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Average Calculation");

    job.setJarByClass(AverageDriver.class);

    // Input and output paths
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    // Setting mapper and reducer classes
    job.setMapperClass(AverageMapper.class);
    job.setReducerClass(AverageReducer.class);

    // Output key and value types
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    // Submitting the job and waiting for it to complete
    System.exit(job.waitForCompletion(true) ? 0 : 1); }
}
```

Mapper Code

```
package temp; import

java.io.IOException;

import org.apache.hadoop.io.IntWritable; import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text; import org.apache.hadoop.mapreduce.Mapper;
public class AverageMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
public static final int MISSING = 9999;

    @Override
    public void map(LongWritable key, Text value, Context context) throws
        IOException, InterruptedException {

        String line = value.toString();

        // Extract year from fixed position
        String year = line.substring(15, 19);
        int temperature;

        // Determine if there's a '+' sign if
        (line.charAt(87) == '+') {
            temperature = Integer.parseInt(line.substring(88, 92));
        } else { temperature = Integer.parseInt(line.substring(87,
            92));
        }

        // Quality check character
        String quality = line.substring(92, 93);

        // Only emit if data is valid
        if (temperature != MISSING && quality.matches("[01459]")) {
            context.write(new Text(year), new IntWritable(temperature)); }
        }
}
```

Reducer Code

```
package temp;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable; import org.apache.hadoop.io.Text; import
org.apache.hadoop.mapreduce.Reducer; public class AverageReducer extends
Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    public void reduce(Text key, Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException
    {

        int sumTemp = 0; int
        count = 0;

        for (IntWritable value : values) {
            sumTemp += value.get(); count++;
        }

        if (count > 0) {
            int average = sumTemp / count; context.write(key,
            new IntWritable(average)); }
        }
}
```

```
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~$ start-all.sh
WARNING: Attempting to start all Apache Hadoop daemons as hadoop in 10 seconds.
WARNING: This is not a recommended production deployment configuration.
WARNING: Use CTRL-C to abort.
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bmscscse-HP-Elite-Tower-800-G9-Desktop-PC]
Starting resourceManager
Starting nodeManagers
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~$ jps
7656 DataNode
7132 SecondaryNameNode
7638 ResourceManager
8231 Jps
5883 org.eclipse.equinox.launcher_1.6.1000.v20250227-1734.jar
7884 NodeManager
6877 NameNode
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -ls /\
> ^C
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -ls /
Found 4 items
drwxr-xr-x - hadoop supergroup      0 2025-04-15 15:00 /FFF
drwxr-xr-x - hadoop supergroup      0 2025-04-15 15:34 /LLL
drwxr-xr-x - hadoop supergroup      0 2024-05-13 14:46 /file
drwxr-xr-x - hadoop supergroup      0 2024-05-13 15:10 /newDataFlair
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -ls /weather
ls: '/weather': No such file or directory
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -mkdir /weather
hadoop@bmscscse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -copyFromLocal /home/hadoop/Desktop/1901.txt /weather/test.txt
```

18 | Page

```

2025-05-06 14:59:24,581 INFO mapreduce.Job: Counters: 36
  File System Counters
    FILE: Number of bytes read=153118
    FILE: Number of bytes written=1493804
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=1776380
    HDFS: Number of bytes written=8
    HDFS: Number of read operations=15
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=4
    HDFS: Number of bytes read erasure-coded=0
  Map-Reduce Framework
    Map input records=6565
    Map output records=6564
    Map output bytes=59076
    Map output materialized bytes=72210
    Input split bytes=103
    Combine input records=0
    Combine output records=0
    Reduce input groups=1
    Reduce shuffle bytes=72210
    Reduce input records=6564
    Reduce output records=1
    Spilled Records=13128
    Shuffled Maps =1
    Failed Shuffles=0
    Merged Map outputs=1
    GC time elapsed (ms)=0
    Total committed heap usage (bytes)=1266679808
  Shuffle Errors
    BAD_ID=0
    CONNECTION=0
    IO_ERROR=0
    WRONG_LENGTH=0
    WRONG_MAP=0
    WRONG_REDUCE=0
  File Input Format Counters
    Bytes Read=888190
  File Output Format Counters
    Bytes Written=8

```

```

    Bytes Written=8
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -ls /weather
Found 2 items
drwxr-xr-x  - hadoop supergroup          0 2025-05-06 14:59 /weather/output
-rw-r--r--  1 hadoop supergroup    888190 2025-05-06 14:50 /weather/test.txt
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -ls /weather/output
Found 2 items
-rw-r--r--  1 hadoop supergroup          0 2025-05-06 14:59 /weather/output/_SUCCESS
-rw-r--r--  1 hadoop supergroup          8 2025-05-06 14:59 /weather/output/part-r-00000
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hadoop fs -cat /weather/output/part-r-00000
1901    46
hadoop@bmscecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ 

```

CODE, COMMAND WITH OUTPUT – B

Driver Code

```
package meanmax;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class MeanMaxDriver { public static void main(String[]
args) throws Exception {

    if (args.length != 2) {
        System.err.println("Please enter both input and output parameters.");
        System.exit(-1);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Mean and Max Temperature");

    job.setJarByClass(MeanMaxDriver.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    job.setMapperClass(MeanMaxMapper.class);
    job.setReducerClass(MeanMaxReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}
```

Mapper Code

```
package meanmax; import
java.io.IOException;

import org.apache.hadoop.io.IntWritable; import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text; import org.apache.hadoop.mapreduce.Mapper; public
class MeanMaxMapper extends Mapper<LongWritable, Text, Text, IntWritable> { public
static final int MISSING = 9999;

    @Override
    public void map(LongWritable key, Text value, Context context) throws
        IOException, InterruptedException {

        String line = value.toString();

        // Extract month from positions 19-20
        String month = line.substring(19, 21);
        int temperature;
```

```

// Extract temperature considering optional '+' if
(line.charAt(87) == '+') {
    temperature = Integer.parseInt(line.substring(88, 92));
} else { temperature = Integer.parseInt(line.substring(87,
92)); }

// Quality check
String quality = line.substring(92, 93);

if (temperature != MISSING && quality.matches("[01459]")) {
    context.write(new Text(month), new IntWritable(temperature)); }
}
}

```

Reducer Code

```

package meanmax; import

java.io.IOException;

import org.apache.hadoop.io.IntWritable; import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer; public class MeanMaxReducer
extends Reducer<Text, IntWritable, Text, Text> {

    @Override
    public void reduce(Text key, Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException {

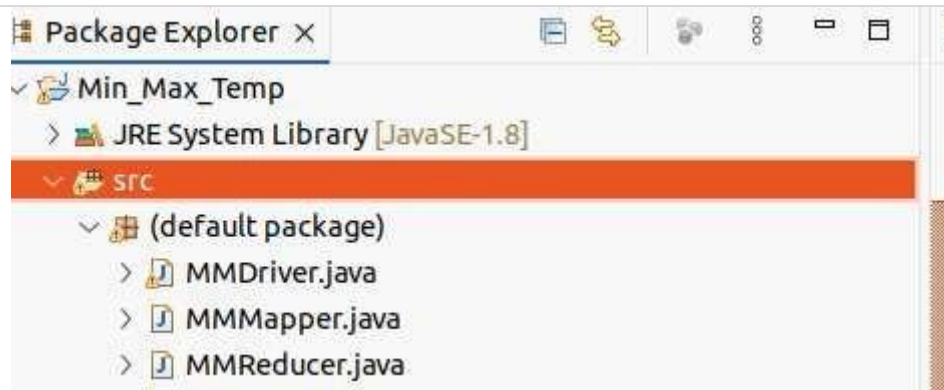
        int sumTemp = 0;
        int count = 0;
        int maxTemp = Integer.MIN_VALUE;

        for (IntWritable value : values) {
            int temp = value.get();
            sumTemp += temp;
            count++;

            if (temp > maxTemp) {
                maxTemp = temp;
            }
        }

        if (count > 0) {
            int avgTemp = sumTemp / count;
            String result = "mean=" + avgTemp + " max=" + maxTemp;
            context.write(key, new Text(result));
        }
    }
}

```



22 | Page

```

2025-05-06 15:26:36,233 INFO mapreduce.Job: Counters: 36
  File System Counters
    FILE: Number of bytes read=126914
    FILE: Number of bytes written=1466688
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=1776388
    HDFS: Number of bytes written=74
    HDFS: Number of read operations=15
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=4
    HDFS: Number of bytes read erasure-coded=0
  Map-Reduce Framework
    Map input records=6565
    Map output records=6564
    Map output bytes=45948
    Map output materialized bytes=59082
    Input split bytes=95
    Combine input records=0
    Combine output records=0
    Reduce input groups=12
    Reduce shuffle bytes=59082
    Reduce input records=6564
    Reduce output records=12
    Spilled Records=13128
    Shuffled Maps =1
    Failed Shuffles=0
    Merged Map outputs=1
    GC time elapsed (ms)=0
    Total committed heap usage (bytes)=1052770304
  Shuffle Errors:
    BAD_ID=0
    CONNECTION=0
    IO_ERROR=0
    WRONG_LENGTH=0
    WRONG_MAP=0
    WRONG_REDUCE=0
  File Input Format Counters
    Bytes Read=888190
  File Output Format Counters
    Bytes Written=74

```

```

hadoop@bnscecse-HP-Elite-Tower-800-G9-Desktop-PC:~$ hdfs dfs -cat /out8/*

```

```

01      4
02      0
03      7
04     44
05    100
06    168
07    219
08    198
09    141
10    100
11     19
12      3

```

LABORATORY PROGRAM – 7

For a given Text file, Create a Map Reduce program to sort the content in an alphabetic order listing only top 10 maximum occurrences of words.

CODE, COMMAND WITH OUTPUT

Driver Code (TopNDriver.java)

```
package samples.topn;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;      import
org.apache.hadoop.mapreduce.lib.input.FileInputFormat; import
org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class TopNDriver {

    public static void main(String[] args) throws Exception {
        if (args.length != 3) {
            System.err.println("Usage:    TopNDriver    <in>    <temp-out>    <final-out>");
            System.exit(2);
        }

        Configuration conf = new Configuration();

        // === Job 1: Word Count ===
        Job wcJob = Job.getInstance(conf, "word count");
        wcJob.setJarByClass(TopNDriver.class);
        wcJob.setMapperClass(WordCountMapper.class);
        wcJob.setCombinerClass(WordCountReducer.class);
        wcJob.setReducerClass(WordCountReducer.class);
        wcJob.setOutputKeyClass(Text.class);
        wcJob.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(wcJob, new Path(args[0]));
        Path tempDir = new Path(args[1]);
        FileOutputFormat.setOutputPath(wcJob, tempDir);

        if (!wcJob.waitForCompletion(true)) {
            System.exit(1);
        }

        // === Job 2: Top N ===
        Job topJob = Job.getInstance(conf, "top 10 words");
        topJob.setJarByClass(TopNDriver.class);
        topJob.setMapperClass(TopNMapper.class);
        topJob.setReducerClass(TopNReducer.class);
        topJob.setMapOutputKeyClass(IntWritable.class);
        topJob.setMapOutputValueClass(Text.class);
        topJob.setOutputKeyClass(Text.class);
        topJob.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(topJob, tempDir);
        FileOutputFormat.setOutputPath(topJob, new Path(args[2]));

        System.exit(topJob.waitForCompletion(true) ? 0 : 1);
    }
}
```

```

    }
}

```

Mapper Code (WordCountMapper.java)

```

package samples.topn;

import java.io.IOException; import
java.util.StringTokenizer;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text; import
org.apache.hadoop.mapreduce.Mapper;

public class WordCountMapper
    extends Mapper<Object, Text, Text, IntWritable> {

    private final static IntWritable ONE = new IntWritable(1);
    private Text word = new Text(); // characters to normalize
    into spaces
    private String tokens = "[_!$%&'\\";

    @Override
    protected void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        // clean & tokenize
        String clean = value.toString()
            .toLowerCase()
            .replaceAll(tokens, " ");
        StringTokenizer itr = new StringTokenizer(clean);
        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken().trim());
            context.write(word, ONE);
        }
    }
}

```

Mapper Code (TopNMapper.java)

```

package samples.topn; import

java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class TopNMapper
    extends Mapper<Object, Text, IntWritable, Text> {

    private IntWritable count = new IntWritable(); private
    Text word = new Text();

    @Override
    protected void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        // input line: word \t count
        String[] parts = value.toString().split("\t"); if
        (parts.length == 2) {
            word.set(parts[0]);
            count.set(Integer.parseInt(parts[1])); // emit
            count → word, so Hadoop sorts by count
            context.write(count, word);
        }
    }
}

```

Reducer Code (WordCountReducer.java)

```
package samples.topn; import  
java.io.IOException;
```

```

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text; import
org.apache.hadoop.mapreduce.Reducer;

public class WordCountReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}

```

Reducer Code (TopNReducer.java)

```

package samples.topn;

import
java.io.IOException;
import java.util.ArrayList;
import
java.util.Collections;
import java.util.List; import
java.util.Map;      import
java.util.TreeMap;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text; import
org.apache.hadoop.mapreduce.Reducer;

public class TopNReducer
    extends Reducer<IntWritable, Text, Text, IntWritable> {

    // TreeMap with descending order of keys (counts) private
    TreeMap<Integer, List<String>>> countMap =
        new TreeMap<>(Collections.reverseOrder());

    @Override
    protected void reduce(IntWritable key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {

        int cnt = key.get();
        List<String> words = countMap.getOrDefault(cnt, new ArrayList<>()); for
        (Text w : values) {
            words.add(w.toString());
        }
        countMap.put(cnt, words);
    }

    @Override
    protected void cleanup(Context context) throws
        IOException, InterruptedException {

        // collect top 10 word→count pairs
        List<WordCount> topList = new ArrayList<>();
        int seen = 0;
        for (Map.Entry<Integer, List<String>>> entry : countMap.entrySet()) {
            int cnt = entry.getKey(); for
            (String w : entry.getValue()) {
                topList.add(new WordCount(w, cnt));
                seen++;
            }
        }
    }
}

```

```
        if (seen == 10) break;
    }
    if (seen == 10) break;

}

// sort these 10 entries alphabetically by word
Collections.sort(topList, (a, b) -> a.word.compareTo(b.word));

// emit final top 10 in alphabetical order for
(WordCount wc : topList) {
    context.write(new Text(wc.word), new IntWritable(wc.count));
}
}

// helper class
private static class WordCount {
    String word; int
    count;
    WordCount(String w, int c) { word = w; count = c; }
}
}
```

```

C:\hadoop-3.3.0\sbin>jps
11072 DataNode
20528 Jps
5620 ResourceManager
15532 NodeManager
6140 NameNode

C:\hadoop-3.3.0\sbin>hdfs dfs -mkdir /input_dir

C:\hadoop-3.3.0\sbin>hdfs dfs -ls /
Found 1 items
drwxr-xr-x - Anusree supergroup 0 2021-05-08 19:46 /input_dir

C:\hadoop-3.3.0\sbin>hdfs dfs -copyFromLocal C:\input.txt /input_dir

C:\hadoop-3.3.0\sbin>hdfs dfs -ls /input_dir
Found 1 items
-rw-r--r-- 1 Anusree supergroup 36 2021-05-08 19:48 /input_dir/input.txt

C:\hadoop-3.3.0\sbin>hdfs dfs -cat /input_dir/input.txt
hello
world
hello
hadoop
bye

```

```

C:\hadoop-3.3.0\hadoop>hadoop jar C:\sort.jar samples.topn.TopN /input_dir/input.txt /output_dir
2021-05-08 19:54:54,582 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2021-05-08 19:54:55,291 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/Anusree/.staging/job_1620483374279_0001
2021-05-08 19:54:55,821 INFO input.FileInputFormat: Total input files to process : 1
2021-05-08 19:54:56,261 INFO mapreduce.JobSubmitter: number of splits:1
2021-05-08 19:54:56,552 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1620483374279_0001
2021-05-08 19:54:56,552 INFO mapreduce.JobSubmitter: Executing with tokens: []
2021-05-08 19:54:56,843 INFO conf.Configuration: resource-types.xml not found
2021-05-08 19:54:56,843 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2021-05-08 19:54:57,387 INFO impl.YarnClientImpl: Submitted application application_1620483374279_0001
2021-05-08 19:54:57,587 INFO mapreduce.Job: The url to track the job: http://LAPTOP-JG329ESD:8088/proxy/application_1620483374279_0001/
2021-05-08 19:54:57,508 INFO mapreduce.Job: Running job: job_1620483374279_0001
2021-05-08 19:55:13,792 INFO mapreduce.Job: Job job_1620483374279_0001 running in uber mode : false
2021-05-08 19:55:13,794 INFO mapreduce.Job: map 0% reduce 0%
2021-05-08 19:55:20,020 INFO mapreduce.Job: map 100% reduce 0%
2021-05-08 19:55:27,116 INFO mapreduce.Job: map 100% reduce 100%
2021-05-08 19:55:33,199 INFO mapreduce.Job: Job job_1620483374279_0001 completed successfully
2021-05-08 19:55:33,334 INFO mapreduce.Job: Counters: 54
File System Counters:
FILE: Number of bytes read=65
FILE: Number of bytes written=530397
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=142
HDFS: Number of bytes written=31
HDFS: Number of read operations=8
HDFS: Number of large read operations=0
HDFS: Number of write operations=2
HDFS: Number of bytes read erasure-coded=0

```

```

C:\hadoop-3.3.0\sbin>hdfs dfs -cat /output_dir/*
hello      2
hadoop     1
world      1
bye        1

C:\hadoop-3.3.0\sbin>

```


LABORATORY PROGRAM – 8

Write a Scala program to print numbers from 1 to 100 using for loop.

CODE, COMMAND WITH OUTPUT

```
scala> for(i <- 1 to 100){  
  | println(i)}  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18
```

Using RDD and FlatMap count how many times each word appears in a file and write out a list of words whose count is strictly greater than 4 using Spark.

CODE, COMMAND WITH OUTPUT

```
scala> val rdd = spark.sparkContext.textFile("file:/home/bmscecse/Desktop/scala")
rdd: org.apache.spark.rdd.RDD[String] = file:/home/bmscecse/Desktop/scala MapPartitionsRDD[1] at textFile at <console>:23

scala> val counts = rdd.flatMap(_.split("\\s+")).map(word => (word.toLowerCase, 1)).reduceByKey(_ + _).filter(_._2 > 4)
counts: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[5] at filter at <console>:25

scala> counts.collect().foreach{ case (word, count) => println(s"$word $count") }
spark 6

scala>
```

Write a simple streaming program in Spark to receive text data streams on a particular port, perform basic text cleaning (like white space removal, stop words removal, lemmatization, etc.), and print the cleaned text on the screen. (Open Ended Question).

CODE, COMMAND WITH OUTPUT

```
# Install NLTK and download required data (run once) !pip
install nltk

import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')

from pyspark.sql import SparkSession from pyspark.sql.functions import col,
lower, regexp_replace, split, explode, udf from pyspark.sql.types import
ArrayType, StringType from pyspark.ml.feature import StopWordsRemover
from nltk.stem import WordNetLemmatizer

# Initialize SparkSession spark =
SparkSession.builder.appName("TextProcessing").getOrCreate()

# Define your input lines
lines = [
    "Hello, I hate you.",
    "I hate that I love you.",
    "Don't want to, but I can't put",
    "nobody else above you."
]

# Create DataFrame from lines df =
spark.createDataFrame(lines, "string").toDF("value")

# Step 1: Lowercase and remove punctuation df_clean =
df.select(regexp_replace(lower(col("value")), "[^a-zA-Z\\s]", "").alias("cleaned"))

# Step 2: Tokenize the cleaned text df_tokens =
df_clean.select(split(col("cleaned"), "\\s+").alias("tokens"))

# Step 3: Remove stop words remover =
StopWordsRemover(inputCol="tokens", outputCol="filtered")
df_filtered = remover.transform(df_tokens)

# Step 4: Lemmatization using NLTK WordNetLemmatizer with UDF lemmatizer
= WordNetLemmatizer()

def lemmatize_words(words):
    return
    [lemmatizer.lemmatize(word) for word in words] lemmatize_udf
= udf(lemmatize_words, ArrayType(StringType()))

df_lemmatized = df_filtered.withColumn("lemmatized", lemmatize_udf(col("filtered")))
```

```
# Step 5: Explode the lemmatized words and show results
df_lemmatized.select(explode(col("lemmatized")).alias("word")).show(truncate=False)
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nltk) (8.2.0)
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from nltk) (1.5.0)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.11/dist-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from nltk) (4.67.1)
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
+-----+
|word |
+-----+
|hello |
|hate |
|hate |
|love |
|dont |
|want |
|cant |
|put |
|nobody|
|else |
+-----+
```

